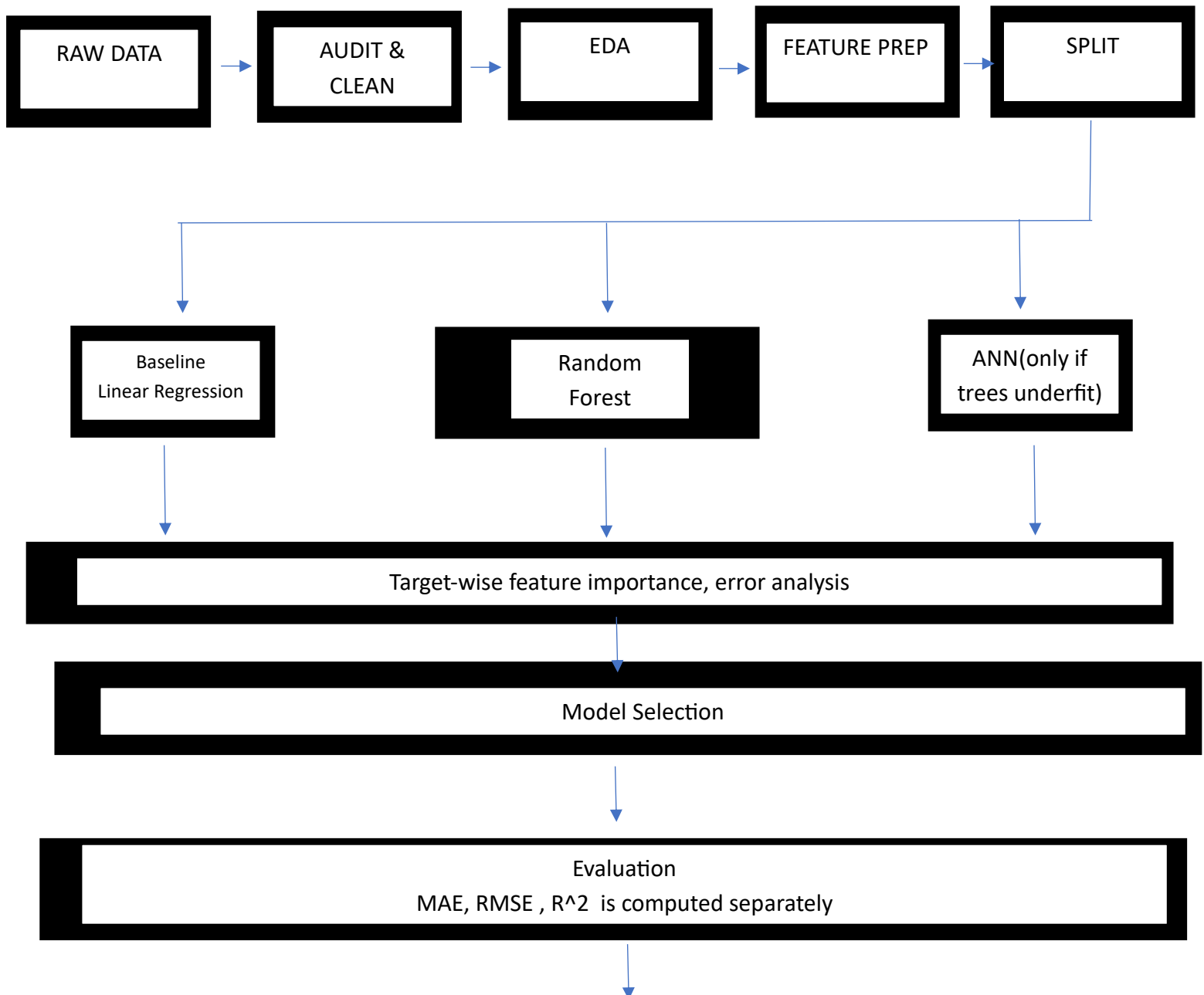


PIPELINE & EXPERIMENT PLANNING

Reproducible Workflow:





Failure & Robustness Analysis



Documentation and final report

- i. Data audit:- understand exactly what arrived before touching it
- ii. Cleaning & preprocessing:- produces model ready table without changing information
- iii. EDA:- understands distributions and relationships before modelling
- iv. Feature Preparation:- build the shared feature set both targets will use
- v. Split:- create a leakage safe train/validation/test split
- vi. Baseline:- fitting linear regression on the shared split
- vii. Advanced models:- compare both random forest and xgboost on identical conditions
- viii. Project specific experiment:- target wise feature importance, error analysis, by changing phase
- ix. Model selection:- choose the final method on evidence, compare models on validation metrics and project specific analysis
- x. Evaluation:- evaluate the selected model on conducting a test; MAE, RMSE, R^2 separately for SoC and remaining time. Report the number that actually matters.
- xi. Failure and robustness analysis:- Inspect worst predictions, check robustness across conditions, understand where and why the model is wrong

Experiment Plan

Baseline Model:

Linear regression (independently trained for SoC & remaining time), using our shared feature engineering set as input.

This doesn't have to be great - just needs to be beaten by all subsequent models. This will also be our very first logged result!

Advances Models

- ❖ Random Forest Regressor : Our primary tree-ensembles candidate. This approach has been shown to be robust to noisy sensor data and nonlinearities inherent. It's also been demonstrated to reliably outperform both linear and neural baselines on this kind of tabular sensor data based on available literature.
- ❖ XGBoost Regressor: Second tree-ensembles candidate that regularly achieves top performance for tabular data types in the literature surveyed. We've included direct comparisons against RF on our same splits.
- ❖ Conditional ANN – Only trained when either RF/XGboost were demonstrably underfitting (as was occasionally observed at distorted temperature profiles/charge cycle extremes flagged in literature review).

Project Specific Experiment

- a. Target-wise feature importance: this was calculated both for SoC and remaining time models, as they're likely to care about very different things (eg temperature is probably much more important for the estimate of remaining time than the current state-of-charge).
- b. Error analysis by charging phase & SoC band: we'll recompute our various metrics for various levels of SoC (low = 0-10%, mid=10-90%, high=90-100%), and also split them up based on the early/mid/late phases of charge to see if the overall picture hides bad performance at the extremes (which the literature review suggested that tree ensembles do).

Model Selection

The final model is chosen using validation-set evidence and the project-specific error analysis, never by repeatedly checking the test set and picking whichever run scores best there. The test set is touched exactly once, for the final evaluation.

Tentative Evaluation Criteria

TARGET	METRIC
SoC	MAE
	RMSE
	R^2
Remaining Time	MAE
	RMSE
	R^2

Experiment log (Log Template)

- i. experiment_id — short unique ID, never reused.
- ii. date, owner — when the run happened and who ran it.
- iii. model — linear_regression / random_forest / xgboost / ann.
- iv. split_version — which recorded train/val/test split definition was used.
- v. target — SoC_percent or remaining_time_min, logged as separate rows per experiment.
- vi. MAE, RMSE, R^2 — metric values (validation set, unless marked as the final test-set row).
- vii. hyperparameters — short string/JSON of what was set for this run.
- viii. notes — anything affecting interpretation (data version, bug fixed, why a run was discarded).
- ix. status — planned / running / done / discarded.

Repository/ Folder Structure

A ready to use repository scaffold is provided

```
|— data/          raw/ (untouched) · interim/ · processed/ (versioned)
|— notebooks/    01_data_audit ... 10_final_evaluation (numbered)
|— src/          data/ · features/ · models/ · evaluation/ · visualization/
|— experiments/  configs/ (one file per run) · logs/ (experiment_log.csv)
|— models/       baseline/ · random_forest/ · xgboost/ ·
ann_conditional/
|— reports/      figures/ · progress_updates/ · final_report/
|— docs/         folder_structure.md and other project docs
|— tests/        lightweight checks (schema validation, split-leakage)
|— README.md · requirements.txt · .gitignore
```

- Everything in data/raw/ is read-only by convention. All other files derive from that dataset. You can always drop everything else and just start over with the full pipeline.
- notebook/ are numbered. When someone opens up my repository (say, a potential mentor who wants to do some spot checking), they'll know the run order immediately without having to ask questions.
- You want experiments/logs/experiment_log.csv to be your single source of truth. Every number you see in the report/presentation should map back to an entry in that file.
- model/ directory is separated by method. This prevents different methods (e.g., baseline, RF, XGboost) from overwriting each other as well as making it easy to reload them separately for failure analysis.